

Домашнее задание по ClickHouse

Основная часть

В ходе выполнения задания были созданы исходная таблица sales, проекция sales_projection и материализованное представление sales_mv с отдельной таблицей хранения агрегатов sales_mv_data.

Проекция хранится внутри исходной таблицы и может использоваться ClickHouse для ускорения запросов, если оптимизатору подходит ее структура. Materialized view в ClickHouse обычно записывает результат в отдельную target-таблицу и переносит часть вычислений на момент вставки данных.

1. Создание таблицы и проекции

```
CREATE TABLE sales
(
    id UInt32,
    product_id UInt32,
    quantity UInt32,
    price Float32,
    sale_date DateTime,

    PROJECTION sales_projection
    (
        SELECT
            product_id,
            sum(quantity) AS total_quantity,
            sum(quantity * price) AS total_sales
        GROUP BY product_id
    )
)
ENGINE = MergeTree
ORDER BY (product_id, sale_date, id);
```

2. Тестовые данные

```
INSERT INTO sales (id, product_id, quantity, price, sale_date) VALUES
(1, 101, 2, 10.00, '2026-05-01 10:00:00'),
(2, 101, 1, 12.50, '2026-05-01 11:00:00'),
(3, 102, 5, 7.20, '2026-05-01 12:00:00'),
(4, 103, 3, 20.00, '2026-05-02 09:00:00'),
(5, 102, 2, 7.00, '2026-05-02 10:00:00'),
(6, 101, 4, 11.00, '2026-05-03 14:00:00'),
(7, 103, 1, 19.50, '2026-05-03 15:00:00'),
(8, 104, 6, 3.30, '2026-05-04 08:00:00');
```

3. Создание materialized view

```
CREATE TABLE sales_mv_data
(
```

```
product_id UInt32,  
total_quantity UInt64,  
total_sales Float64  
)  
ENGINE = SummingMergeTree  
ORDER BY product_id;  
  
CREATE MATERIALIZED VIEW sales_mv  
TO sales_mv_data  
AS  
SELECT  
product_id,  
sum(quantity) AS total_quantity,  
sum(quantity * price) AS total_sales  
FROM sales  
GROUP BY product_id;
```

4. Запросы к данным

Запрос к исходной таблице sales:

```
SELECT  
product_id,  
sum(quantity) AS total_quantity,  
sum(quantity * price) AS total_sales  
FROM sales  
GROUP BY product_id  
ORDER BY product_id;
```

Запрос к materialized view:

```
SELECT  
product_id,  
total_quantity,  
total_sales  
FROM sales_mv_data  
ORDER BY product_id;
```

5. Сравнение производительности

Для сравнения производительности были использованы показатели query_duration_ms, read_rows и read_bytes из system.query_log.

```
SYSTEM FLUSH LOGS;  
  
SELECT  
query,  
query_duration_ms,  
read_rows,  
read_bytes,  
projections  
FROM system.query_log  
WHERE type = 'QueryFinish'
```

```
AND query LIKE 'SELECT%'
ORDER BY event_time_microseconds DESC
LIMIT 20;
```

В результате было видно, что запрос к sales использовал проекцию sales_projection, что подтверждается значением projections = ['default.sales.sales_projection']. Запрос к sales_mv_data выполнялся без использования проекции, так как данные уже хранятся в заранее агрегированном виде.

Как правило, запрос к исходной таблице читает больше строк и байт, запрос с проекцией выполняется быстрее, а запрос к sales_mv_data обычно является самым дешевым по чтению.

Задание со звездочкой

1. Проверка влияния вставки новых данных

```
INSERT INTO sales (id, product_id, quantity, price, sale_date)
VALUES (9, 101, 3, 13.00, now());
```

```
SELECT
  product_id,
  sum(quantity) AS total_quantity,
  sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id
ORDER BY product_id;
```

```
SELECT
  product_id,
  total_quantity,
  total_sales
FROM sales_mv_data
ORDER BY product_id;
```

2. Проверка влияния изменения и удаления строк

```
ALTER TABLE sales UPDATE price = 15.00 WHERE id = 1;
ALTER TABLE sales DELETE WHERE id = 2;
```

После этого можно повторно выполнить агрегирующие запросы и сравнить результат. Это показывает, как изменения в исходной таблице влияют на агрегаты.

3. Вариант с другими агрегатными функциями

```
CREATE TABLE sales_mv_data_2
(
  product_id UInt32,
  cnt UInt64,
  avg_price Float64,
  max_price Float32
)
ENGINE = SummingMergeTree
```

```
ORDER BY product_id;

CREATE MATERIALIZED VIEW sales_mv_2
TO sales_mv_data_2
AS
SELECT
    product_id,
    count() AS cnt,
    avg(price) AS avg_price,
    max(price) AS max_price
FROM sales
GROUP BY product_id;
```

Краткий вывод

Проекция удобна для ускорения запросов внутри исходной таблицы без создания отдельной структуры хранения. Materialized view полезно, когда нужен отдельный агрегированный слой данных. В данном задании это подтверждается тем, что запрос к sales использует проекцию, а запрос к sales_mv_data читает уже готовые агрегаты.

Источники

- [ClickHouse documentation: Projections](#)
- [ClickHouse documentation: Query log](#)
- [ClickHouse documentation: Materialized views](#)